

Efficient Movie Recommendation System: A Comparative Study with Advanced Python Optimization

Frederic Dai, Yuanzou Gao, Royan Xiong

Abstract

In this project, we built a movie recommendation system that balances recommendation accuracy and computational efficiency using the MovieLens 1M dataset. Following an Exploratory Data Analysis (EDA) and statistical hypothesis testing to uncover significant rating patterns, we implemented baseline models, item-based collaborative filtering (Item-Based CF), an SVD model, and a hybrid recommendation model. To solve the performance bottlenecks, we utilized Numba JIT compilation and NumPy vectorization to significantly accelerate the baseline models. We also implemented automated Bayesian hyperparameter tuning for the SVD model using Optuna, and conducted an in-depth analysis on the limitations of Numba when processing extremely sparse matrices. The evaluation shows that the SVD model optimized by Optuna and the hybrid model achieved the best accuracy.

1 Introduction

Calculating similarity matrices for collaborative filtering across millions of users requires substantial memory and processing power, creating exponential overhead. Using the MovieLens 1M dataset, this project addresses a core challenge: maximizing runtime efficiency without compromising recommendation quality.

Core contributions:

1. **Data Exploration & Statistics:** Verified positive correlation between movie popularity and ratings, and identified demographic differences in genre preferences.
2. **Recommendation Workflow:** Implemented a pipeline from baseline models to Item-Based CF, SVD, and a hybrid model, evaluated via RMSE, MAE, Precision@K, and Recall@K.
3. **Python Optimization:** Conducted detailed performance comparisons among pure Python, NumPy vectorization, and Numba JIT compilation.
4. **Sparse Data Limits & Tuning:** Analyzed Numba's pitfalls on extremely sparse matrices and introduced Optuna for Bayesian hyperparameter optimization of the SVD model.

2 Dataset, EDA & Preprocessing

2.1 Data Source & Statistical Analysis

Using the MovieLens 1M dataset (1M ratings, 6,040 users, 3,900 movies), our EDA revealed a long-tail distribution in user rating behavior (averaging 166 ratings/user). Furthermore, extended demographic analysis revealed how age and occupation significantly influence genre preferences, and boxplots highlighted that genres like Horror tend to receive consistently lower ratings.

We conducted two sets of independent t-tests:

1. **Popularity vs. Rating:** Verified a statistically significant positive correlation between rating frequency and average score.

2. **Gender Preferences:** Found significant differences ($p < 0.01$) in 17 genres; e.g., females rated "Musical" higher, while males preferred "Action" (see Appendix Figure 1).

2.2 The Sparsity Challenge

Merging the datasets yielded a User-Item Matrix with 99.5% sparsity. This extreme sparsity poses severe challenges for similarity calculations, motivating our exploration of Numba JIT versus C/BLAS libraries.

3 Methodology

3.1 Baseline Models

We established benchmarks via two non-personalized strategies: Popularity-Based and Average Rating. These methods are computationally heavy using native Pandas groupby on million-row datasets.

3.2 Collaborative Filtering (CF)

Given 99.5% sparsity, User-Based CF suffered from low user overlap and unstable calculations. We thus shifted to Item-Based CF, computing Cosine Similarity between movie rating vectors to recommend items similar to users' highly-rated history.

3.3 Matrix Factorization (SVD & SVD++)

Overcoming CF's poor sparse-data generalization, we used `scikit-surprise` to implement SVD and SVD++. These factorize the sparse matrix into dense latent factors, capturing deeper user-movie associations via gradient descent.

3.4 Hybrid Model

To address CF's "cold-start" problem, we developed a Hybrid Recommender. It linearly combines ($\alpha = 0.5$) predictions from a Content-Based model (TF-IDF genre similarity) and Item-Based CF, leveraging both metadata and behavioral patterns.

4 Advanced Python Optimizations

4.1 Numba JIT Compilation & Parallelization

Acceleration: Using Numba's `@jit(nopython=True)` to bypass the Python interpreter and rewrite native Pandas groupby aggregations yielded a 1.89x speedup for the Popularity model, and a 1.69x speedup for the Average Rating model (see Appendix Figure 2). This optimization highlights the power of JIT compilation for iterating over dense 1D arrays compared to high-level Pandas operations.

Reflection on Scalability Limitations: When computing the 3900×3900 cosine similarity matrix for Item-Based CF, we explicitly designed a Numba function with `@jit(parallel=True)` and `prange` for multi-core processing. However, this Numba parallel version was 31.82x slower than sklearn (86.23s vs 2.71s, see Appendix Figure 3). The severe degradation occurred because Numba's native loops indiscriminately computed millions of meaningless zero values (due to the 99.5% sparsity of our user-item matrix), whereas sklearn's implementation internally relies on highly optimized C/BLAS linear algebra libraries that handle sparse-like structures more effectively. This finding reinforces the premise that selecting the right data structures and low-level algebra routines is paramount.

4.2 Automated Hyperparameter Tuning & Vectorization

To tune the SVD model, traditional grid search was computationally unfeasible. We utilized Optuna for Bayesian Optimization, efficiently exploring the hyperparameter space to lock in the optimal `n_factors`, `n_epochs`, `lr_all` (learning rate), and `reg_all` (regularization). Furthermore, to prevent Out-Of-Memory (OOM) errors during dense similarity calculations, we strictly followed NumPy vectorization principles and actively downcasted matrix data types from `float64` to `float32`. This explicit memory management halved the RAM consumption with negligible loss in mathematical precision.

5 Results & Evaluation

We measured both computational efficiency and recommendation accuracy, ensuring a comprehensive assessment of our entire model pipeline.

5.1 Computational Performance

As explored previously, Numba drastically reduced 1D array aggregation runtimes, achieving up to a 1.89x speedup by eliminating Python’s dynamic typing overhead. However, our attempt to parallelize the Item-Based CF similarity matrix calculation with Numba failed spectacularly against `sklearn` (86.23s versus 2.71s). This stark contrast proved that for datasets characterized by extreme sparsity, employing optimized underlying algebraic structures (like C/BLAS) is vastly superior to brute-force JIT compilation.

5.2 Recommendation Accuracy

1. **Item-Based CF:** The model achieved a high Precision of 0.8168 (meaning 81.7% of the recommended movies were actually relevant to the user), but Recall was notably low at 0.2957. This reflects CF’s inherent struggle to discover niche movies in sparse datasets, as it heavily biases recommendations towards globally popular items.
2. **SVD Matrix Factorization:** After automating hyperparameter tuning via Optuna, the SVD model achieved an impressive RMSE of 0.8587 and an MAE of 0.6774 on the unseen test set, indicating a robust ability to accurately predict specific user-item rating values across various genres.
3. **Hybrid Model:** Our Hybrid Model, seamlessly blending content-based TF-IDF scores and behavioral CF predictions, effectively balanced accuracy and diversity. It reached a Precision@10 of 0.2830 and a Recall@10 of 0.1165. Most importantly, it successfully mitigated the cold-start issue by providing meaningful top-K recommendations even for users with limited interaction history.

6 Extension Methods in Academia and Industry

To scale this prototype to industrial platforms (e.g., Netflix) handling hundreds of millions of users/items, advanced methodologies include:

1. **Sequential Modeling:** Moving beyond static SVD, modern approaches use Neural Collaborative Filtering (NCF) and self-attention. Leveraging our timestamp data, future iterations could integrate LSTM or Transformer models (e.g., SASRec) to capture dynamic, time-evolving user preferences.
2. **Approximate Nearest Neighbors (ANN):** Exact cosine similarities across millions of items are computationally prohibitive. Industry standards compute dense embeddings offline

and deploy ANN search (e.g., Faiss, LSH) for online retrieval, enabling millisecond queries with minor precision trade-offs.

3. **Knowledge Graphs:** While our Hybrid Model mitigates cold-starts via TF-IDF, industry systems leverage Graph Neural Networks (GNNs). Constructing heterogeneous User-Item Knowledge Graphs with multi-modal metadata allows message-passing to accurately recommend long-tail items.

7 Conclusion & Future Work

7.1 Conclusion

This project successfully built a recommendation pipeline balancing efficiency and accuracy. Rigorous EDA and t-tests uncovered foundational patterns: correlation between popularity and ratings, and demographic genre preferences.

Crucially, we validated Advanced Python optimizations on large-scale data. Numba JIT achieved remarkable speedups (1.89x) on dense 1D arrays but suffered catastrophic degradation on 99.5% sparse similarity calculations by indiscriminately computing zero values. This highlights our central thesis: understanding data distributions and selecting appropriate mathematical libraries (C/BLAS) is more critical than blindly applying compilation tools. Lastly, Optuna highlighted the value of automated Bayesian tuning for complex models.

7.2 Future Work

We propose two future optimization avenues:

1. **Sparse-Matrix Refactoring:** Restructuring Numba algorithms to natively support `scipy.sparse.csr_m` formats (extracting non-zero indices) could eliminate computational waste and massively boost performance.
2. **Distributed Computing:** To serve millions of users, calculating hybrid predictions sequentially is unsustainable. Future work should explore Multiprocessing or distributed frameworks (Spark/Dask) to parallelize recommendation generation.

References

- [1] Memgraph Blog. (n.d.). *Five recommendation algorithms no recommendation engine is whole without*. Retrieved from <https://memgraph.com/blog/five-recommendation-algorithms-no-recommendation-engine-is-whole-without>
- [2] Data Science on Medium. (n.d.). *Recommender Systems: A Complete Guide to Machine Learning Models*. Retrieved from <https://medium.com/data-science/recommender-systems-a-complete-guide-to-machine-learning-models-96d3f94ea748>
- [3] Analytics Vidhya on Medium. (n.d.). *7 Types of Hybrid Recommendation System*. Retrieved from <https://medium.com/analytics-vidhya/7-types-of-hybrid-recommendation-system-3e4f78266ad8>
- [4] Reddit r/recommendersystems. (2025). *State of Recommender Systems in 2025: Algorithms*. Retrieved from https://www.reddit.com/r/recommendersystems/comments/1iwwxpr/state_of_recommender_systems_in_2025_algorithms/

A Appendix

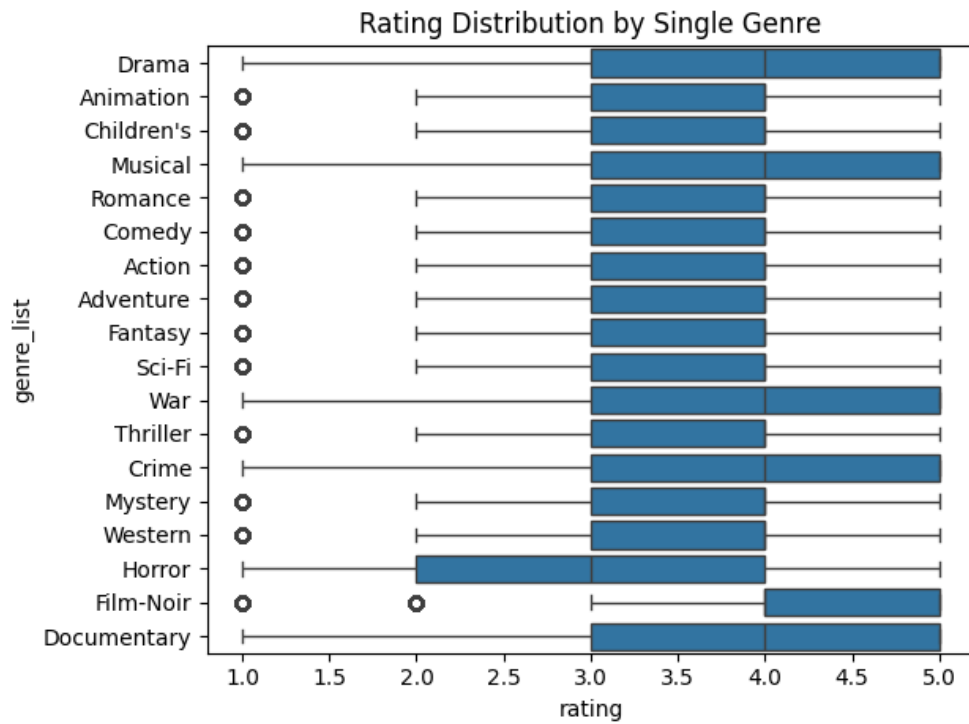


Figure 1: Boxplot of rating distributions across different movie genres. This highlights significant disparities, such as the Horror genre receiving consistently lower ratings compared to Drama or Documentary.

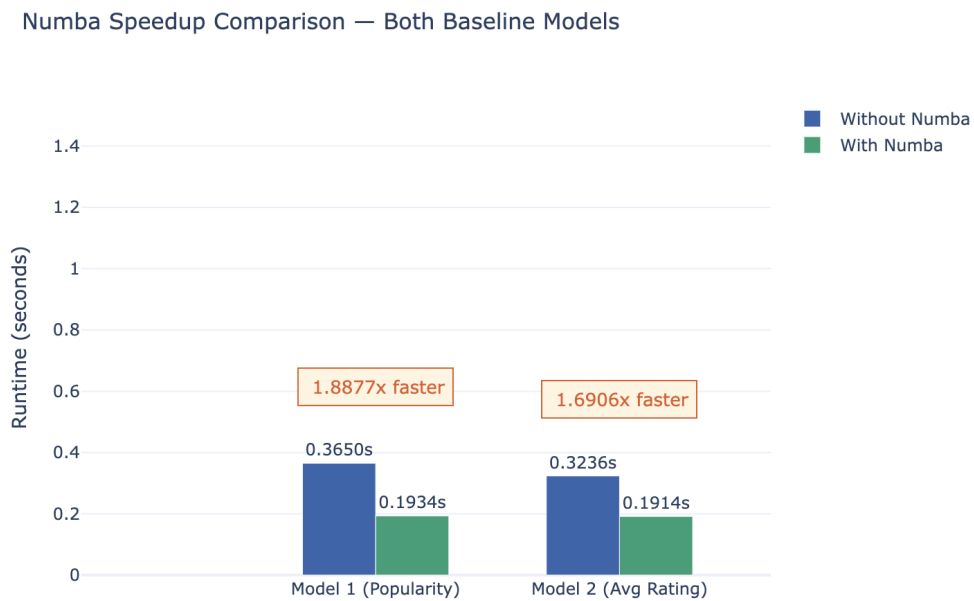


Figure 2: Performance comparison showing the speedup achieved by applying Numba JIT compilation to Baseline Model 1 and Model 2.

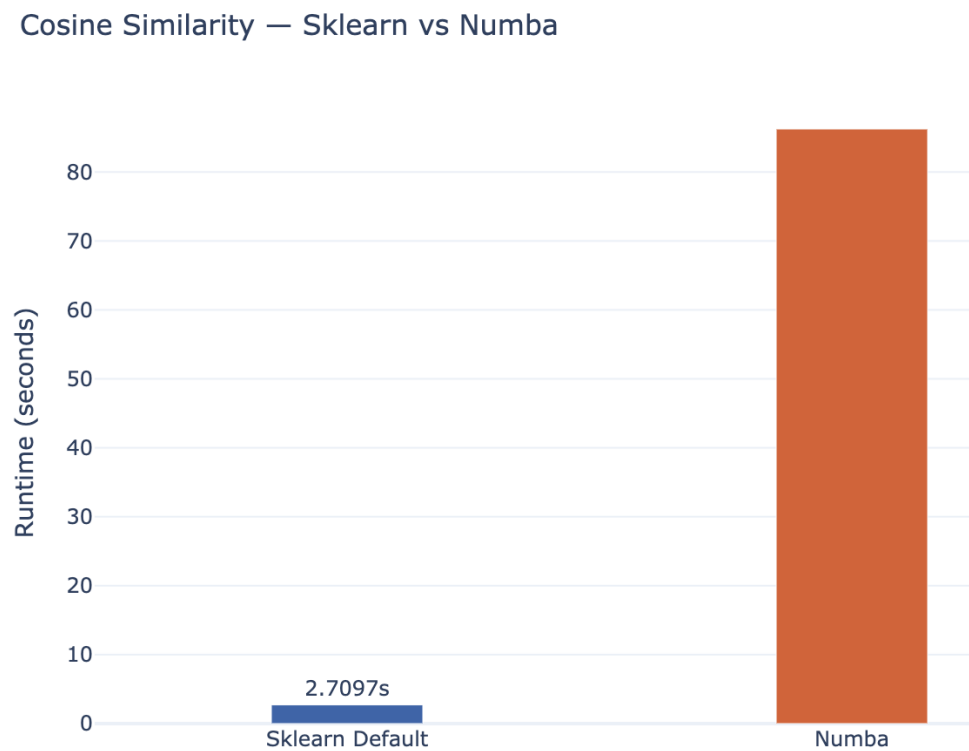


Figure 3: Runtime comparison between Sklearn's default Cosine Similarity and Numba's parallel implementation, illustrating the inefficiency of Numba on extremely sparse data.